LESSON 12

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Bat Ears

Measure distance with sound, then build a car parking sensor.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How the **ultrasonic sensor** uses echoes to measure distance
- How to time a pulse with `pulseIn()`
- Why a **voltage divider** keeps 5 volts away from the ESP32
- How to make a **function** that gives back a value with `return`

SESSION PLAN

0:00	How bats and dolphins "see" with sound
0:15	Build the sensor circuit with the voltage divider
0:35	Sketch 1: **L12_Distance**
0:55	Short break
1:00	Experiments: how far, how small, how soft?
1:15	Sketch 2: **L12_Parking** (add the buzzer)
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Ultrasonic sensor (HC-SR04, the one with two "eyes")
- ☐ 1 resistor, 1 kΩ (brown-black-red)
- ☐ 1 resistor, 2 kΩ (red-black-red)
- ☐ Passive buzzer (green board underneath)
- ☐ Jumper wires
- ☐ Extra: OLED screen from Lesson 10

1. Seeing with sound

Bats hunt in the dark by shouting very high squeaks and listening for the **echo**. The longer the echo takes to come back, the further away the moth is.

The ultrasonic sensor does exactly the same thing. One "eye" (**T**, the transmitter) is a tiny speaker. The other (**R**, the receiver) is a tiny microphone. The sound is **ultrasonic**: 40 000 vibrations per second, far too high for humans (or your dog) to hear.

> **Big idea: distance = speed × time**
>
> Sound travels about **343 metres per second**. That is **0.0343 cm every microsecond** (a microsecond is a millionth of a second). The sound goes *there and back*, so we divide by 2:
>
> **distance in cm = echo time × 0.0343 ÷ 2**

The sensor has 4 pins:

Pin	Job
VCC	Power: needs **5 V**
Trig	"Trigger": we send a short pulse here to say *shout now!*
Echo	The sensor holds this pin HIGH until it hears the echo
GND	Ground

2. Build the circuit

The sensor runs on 5 V, so its **Echo** pin sends back 5 V. That is too much for the ESP32, whose pins only like **3.3 V**. We use two resistors as a **voltage divider**: they share the 5 V between them so only about 3.3 V reaches the ESP32.

> **Don't skip the resistors!**
>
> Connecting Echo straight to the ESP32 can damage pin 16 over time. The divider takes 1 minute to build and keeps your board safe.

✓	From	To
☐	ESP32 **GND**	breadboard **–** rail
☐	ESP32 **5V** (may say **VIN**)	breadboard **+** rail
☐	Sensor **VCC**	**+** rail (5 V)
☐	Sensor **GND**	**–** rail
☐	Sensor **Trig**	ESP32 pin **17**
☐	Sensor **Echo**	an empty breadboard row (call it *row E*)
☐	**1 kΩ** resistor	from *row E* to another empty row (call it *row M*, the middle)
☐	**2 kΩ** resistor	from *row M* to the **–** rail
☐	*row M*	ESP32 pin **16**

> **Check the + rail**
>
> In earlier lessons the **+** rail may have been 3.3 V. Today it is **5 V**. Only the sensor's VCC should use it.

Point the sensor's eyes towards the edge of the table, with nothing in front of it.

3. Sketch 1: Measure the distance

File → New Sketch, type this in and save it as **L12_Distance**. Upload, then open the **Serial Monitor** at 115200 baud.

Type this in → save it as **L12_Distance** 37 lines

```
// Lesson 12: Measure distance with sound
// HC-SR04: TRIG on pin 17, ECHO on pin 16 (through the divider).

const int TRIG_PIN = 17;
const int ECHO_PIN = 16;

float readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);   // a 10 microsecond "ping"
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long echoTime = pulseIn(ECHO_PIN, HIGH, 30000);
  if (echoTime == 0) {
    return -1;                    // no echo came back
  }
  return echoTime * 0.0343 / 2;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

void loop() {
  float cm = readDistance();
  if (cm < 0) {
    Serial.println("Nothing in range");
  } else {
    Serial.print("Distance: ");
    Serial.print(cm);
    Serial.println(" cm");
  }
  delay(200);
}
```

Hold a book in front of the sensor and move it slowly closer and further away.

How it works

Code	What it means
`float readDistance() { … }`	A **function** we made up that *gives back* a `float` (a number with a decimal point)
`delayMicroseconds(10);`	Wait 10 *micro*seconds: 100 000 times shorter than `delay(1000)`!
`digitalWrite(TRIG_PIN, HIGH);`	A tiny pulse that tells the sensor: *shout now!*
`pulseIn(ECHO_PIN, HIGH, 30000)`	Stopwatch: how many microseconds does the Echo pin stay HIGH? Gives up after 30 000 µs
`long echoTime`	A `long` is like an `int`, but it can hold much bigger numbers
`return echoTime * 0.0343 / 2;`	Work out the distance and **send it back** to whoever called the function
`return -1;`	No echo means nothing is in range. We use –1 as a secret code for "*I don't know*"
`float cm = readDistance();`	Call the function and keep its answer in `cm`

Experiments

1. Use a ruler. Is the sensor accurate? How many cm is it off by?

   _____

2. What is the **closest** and **furthest** distance it can measure?

   _____

3. Try a pencil, your hand, a cushion and a wall. Which ones are hard to "see"? Why do you think soft things are tricky?

   _____

   _____

Only seeing 0 or *Nothing in range*?

Check that the sensor's VCC goes to **5V**, not 3V3. Check the divider: Echo → 1 kΩ → row M → 2 kΩ → GND, and row M → pin 16.

4. Sketch 2: Parking sensor

Cars have sensors like this in their bumpers. They beep slowly when a wall is far away, faster as it gets closer, and non-stop when you must STOP!

Add the **passive buzzer** (the one with the green board underneath):

✓	From	To
☐	Buzzer **+** (or **S**)	ESP32 pin **33**
☐	Buzzer **–**	**–** rail

Use **File** → **Save As…** to save a copy of your sketch as **L12_Parking**, then change it to match this:

Type this in → save it as **L12_Parking** 42 lines

```
// Lesson 12: Parking sensor
// Beeps faster as you get closer. Passive buzzer on pin 33.

const int TRIG_PIN = 17;
const int ECHO_PIN = 16;
const int BUZZER_PIN = 33;

float readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long echoTime = pulseIn(ECHO_PIN, HIGH, 30000);
  if (echoTime == 0) {
    return -1;
  }
  return echoTime * 0.0343 / 2;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

void loop() {
  float cm = readDistance();
  Serial.println(cm);

  if (cm < 0 || cm > 50) {
    noTone(BUZZER_PIN);           // far away: stay quiet
    delay(200);
  } else if (cm < 5) {
    tone(BUZZER_PIN, 1000);       // very close: non-stop beep
    delay(100);
  } else {
    tone(BUZZER_PIN, 1000, 60);   // short beep...
    delay(cm * 10);               // ...gap gets shorter when close
  }
}
```

Code	What it means
`cm < 0 || cm > 50`	`||` means **OR**: *nothing in range* OR *further than 50 cm*
`else if (cm < 5)`	Checked only if the first test was false
`tone(BUZZER_PIN, 1000, 60);`	Play a 1000 Hz note for 60 ms
`delay(cm * 10);`	Wait longer when the distance is bigger. 40 cm → 400 ms, 10 cm → 100 ms

Try it

1. Make the buzzer start beeping when you are 1 metre away.
2. Change the beep's **pitch** too, so it gets higher as you get closer.

5. Challenges

Challenge 1: Traffic light distance

Add your red, yellow and green LEDs (pins 25, 26, 27). Green when far, yellow when getting close, red when too close.

Challenge 2: Inches

Add a second function `readInches()` that uses `readDistance()` and divides by 2.54. Print both.

Challenge 3: Air instrument

Make a **theremin**: no beeping, just one note that changes pitch as your hand moves. *Hint:* `map()` *the distance 3–40 cm to a note of 1500–200 Hz.*

Bonus: Screen readout

Connect the OLED from Lesson 10 (SDA 21, SCL 22). Show the distance in big numbers, plus a bar that shrinks as things come closer.

6. Quiz

Check what you learned

1. Why do we divide the echo time by 2?

   _____

2. Why do we need the 1 kΩ and 2 kΩ resistors?

   _____

   _____

3. An echo takes 1000 µs. How far away is the object? (Use the formula!)

   _____

Tidy up: unplug the USB cable. Keep the sensor's divider resistors together so you can find them again.